



Ukrainian Asylum Tracking in Europe (UATE)

WT 23/24 IP – Spatial Data Infrastructure (SDI) Services Implementation

UATE Project Documentation

Paris Lodron University of Salzburg

Department of Geoinformatics (ZGIS)

Student: Md Saleh Shakeel Nomaan

Professor: Manfred Mittlböck

CONTENTS

Abstract	1
Project description	2
Project management.....	2
a) Work breakdown structure	3
WP Concept development.....	3
b) Objective	4
c) Purpose.....	4
d) Resource exploration	4
e) SDI Methods	5
f) SDI Implementation.....	6
g) SDI architecture	6
WP Data collection and integration	7
a) Geocoding the data.....	8
b) Database connection	9
WP 4 Geospatial services and mapping.....	9
a) GeoServer integration	9
b) Setting up Django project and accessing the data.....	10
c) Mapping library integration	11
WP Dashboard development.....	12
a) Origin destination visualization	12
b) Sample chart.....	13
c) User interaction implementation.....	14
d) Final dashboard.....	15

ABSTRACT

The UATE project aimed to develop a spatial data infrastructure (SDI) to track and analyze Ukrainian asylum seekers and refugee migration to Europe. The project was built on Python Django web framework and GeoServer. It integrated geospatial data processing and visualization techniques to offer insights into migration patterns. Project data was sourced from the United Nations High Commissioner for Refugees (UNHCR) and enriched with location information using Google's Geocoding API. The SDI employed Python for backend development and JavaScript for frontend interactivity, leveraging Chart.js library for data visualization and Google Maps JavaScript API as the mapping library. The project data was stored and managed in a PostgreSQL/PostGIS database. GeoServer was used to facilitate publishing of geospatial data as Web Feature Services (WFS) and Web Map Services (WMS), allowing for dynamic data retrieval and mapping from the Django framework. Results from the project demonstrated the power of integrating various open-source and low-cost technologies to create a comprehensive tool for an SDI. The dashboard provides a valuable resource for students, researchers, and policymakers, offering detailed visualizations of migration trends and the ability to drill down into specific data points.

PROJECT DESCRIPTION

This document describes the implementation of a spatial data infrastructure (SDI) for keeping track of the Ukrainian asylum and refugee dynamics in Europe. The infrastructure was designed to be consumed as a dashboard to demonstrate possibilities for the available data. The implementation provides a basis for future integration with other data points to enrich the resource.

The developed dashboard is intended to serve as a resource for understanding the complexities of Ukrainian migration trends, demographic changes, and asylum application dynamics within Europe's refugee landscape. It should be noted that for implementation purposes, sampled European countries were used due to the limitations to fetching the data from the source API. However, a solution could be explored in future to provide an avenue for scalability. For the interest of the project, the data used was from 2013 to 2022. The infrastructure was designed to be implemented with open-source resources. However, slight costs had to be incurred for smoother geocoding using the Google's Geocoding API.

PROJECT MANAGEMENT

A project management strategy was designed for UATE to guide the execution of the project. This involved breaking down the project into smaller working packages and refining the objectives and milestones of each where applicable. It was aided by drafting the following documents:

- Gannt chart
- PERT chart
- Project overview document
- Risk matrix
- Timesheet

The primary project management tool used for the was GitLab and GitHub. It was used to host scripts, metadata files, creation and tracking issues ,and most importantly collaboration within the project team and getting feedback from peers and professors.

Work breakdown structure

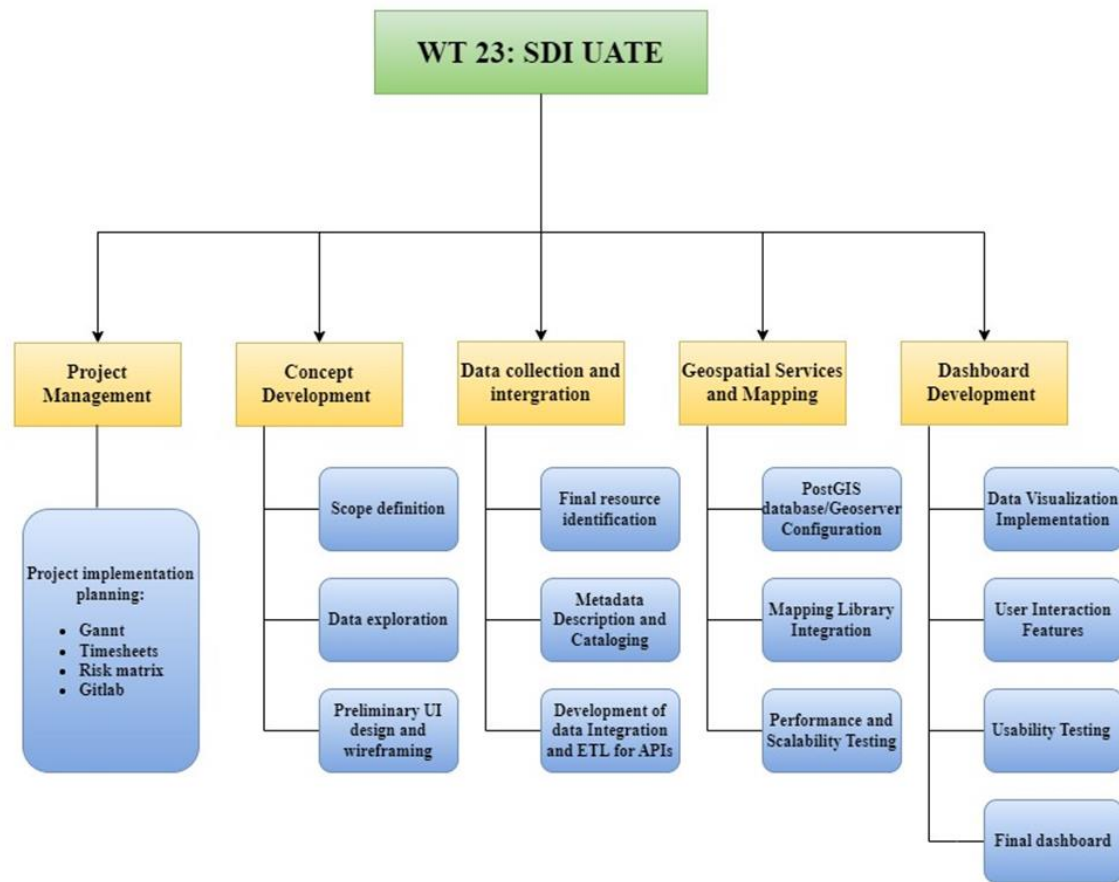


Figure 1: Image showing the project breakdown to work packages.

WP CONCEPT DEVELOPMENT

The primary objective of this work package was to lay out the foundation of the project by defining the project idea, scope and identifying possible stakeholders. Due to the limited timeframe defined for the project, it was refined to focus on Ukrainian asylum seekers in sampled European countries as destinations. The decision was informed by the current war in Ukraine. Therefore, the objective and purpose of the project were defined as described in the subsequent sections.

a) Objective

The primary objective was to develop a spatial data infrastructure (SDI) strategy to track and analyze refugee and asylum movements from Ukraine to European countries. The SDI will be consumed as a dashboard by clients.

b) Purpose

The purpose of the UATE project was to generate heightened awareness and knowledge among students in migration studies. It provides them with insightful, historical, and up-to-date information on Ukrainian asylum dynamics in Europe. Additionally, it aimed at providing temporal aspect which will allow students to track patterns over time. With an education objective in mind during the design phase, UATE shades light into nuanced complexities of migration, enabling students to explore real-time data, understand the broader context, and engage with one of the world's most pressing global challenges. By doing so, UATE would benefit students by offering a unique resource for informed learning.

c) Resource exploration

In the concept definition phase, resource exploration was done to identify potential data resources to be used, how they will be stored, hosted, and retrieved by clients. The resources and services identified were the primary ingredients of the UATE SDI as they informed the and enriched the different components of the infrastructure.

The primary data source for the project was the United Nations High Commissioner for Refugees (UNHCR) refugee statistics API. The API provides access to a wide range of refugee data; however, for the scope of this project only the asylum applications, decisions, and demographic data were utilized. Preliminary test was done to test for the feasibility of the data at the initial stage by retrieving it and storing it in the database for preliminary inspection as shown below:

id integer	year integer	coa_name text	coa_name text	procedure_ty text	dec_level text	dec_recogniz integer	dec_other integer	dec_rejected integer	dec_closed integer	dec_total integer	coa_point geometry
1	2013	Ukraine	Austria	G	FA	0	10	68	10	88	0101000020E61...
2	2015	Ukraine	Austria	G	FA	22	10	115	47	194	0101000020E61...
3	2016	Ukraine	Austria	G	FA	22	22	67	125	236	0101000020E61...
4	2017	Ukraine	Austria	G	FA	10	13	247	183	453	0101000020E61...
5	2018	Ukraine	Austria	G	FA	23	25	260	74	382	0101000020E61...
6	2019	Ukraine	Austria	G	FA	12	10	363	47	432	0101000020E61...
7	2020	Ukraine	Austria	G	FA	10	5	175	19	209	0101000020E61...
8	2021	Ukraine	Austria	G	FA	11	22	100	20	153	0101000020E61...
9	2022	Ukraine	Austria	G	FA	5	75	31	49	160	0101000020E61...
10	2013	Ukraine	Germany	G	NA	0	5	24	26	55	0101000020E61...
11	2014	Ukraine	Germany	G	NA	10	10	36	274	330	0101000020E61...
12	2015	Ukraine	Germany	G	NA	51	0	42	874	967	0101000020E61...
13	2016	Ukraine	Germany	G	NA	28	34	2260	1605	3927	0101000020E61...
14	2017	Ukraine	Germany	G	NA	109	186	4071	1046	5412	0101000020E61...
15	2018	Ukraine	Germany	G	NA	20	53	1107	181	1361	0101000020E61...

Figure 2: Screenshot showing decisions data tables in PostgreSQL/ PostGIS

d) SDI Methods

To build the SDI, commercial off the shelf (COTS) services were identified and used in building the architecture. Additionally, the open geospatial consortium (OGC) methods were implemented as summarized below:

No.	COTS	OGC
1	Google Maps Geocode API	Web Feature Service (WFS)
2	Google Maps JavaScript API	Simple Features

Figure 3: Table showing COTS and OGC services utilized in the project

e) SDI Implementation

The implementation of the infrastructure considered the data storage, hosting, and metadata services as summarized below:

Database	Services	Metadata
PostgreSQL (PostGIS)	• OGC WFS & WMS • GeoServer	ISO 19139 (XML)

Figure 4: Table showing database, services and metadata strategies

f) SDI architecture

Bringing in the various components of the SDI, Python and JavaScript were the primary languages used alongside their respective libraries and framework. A visual representation of the architecture encompassing the different components i.e., services and OGC methods and deployment is shown below:

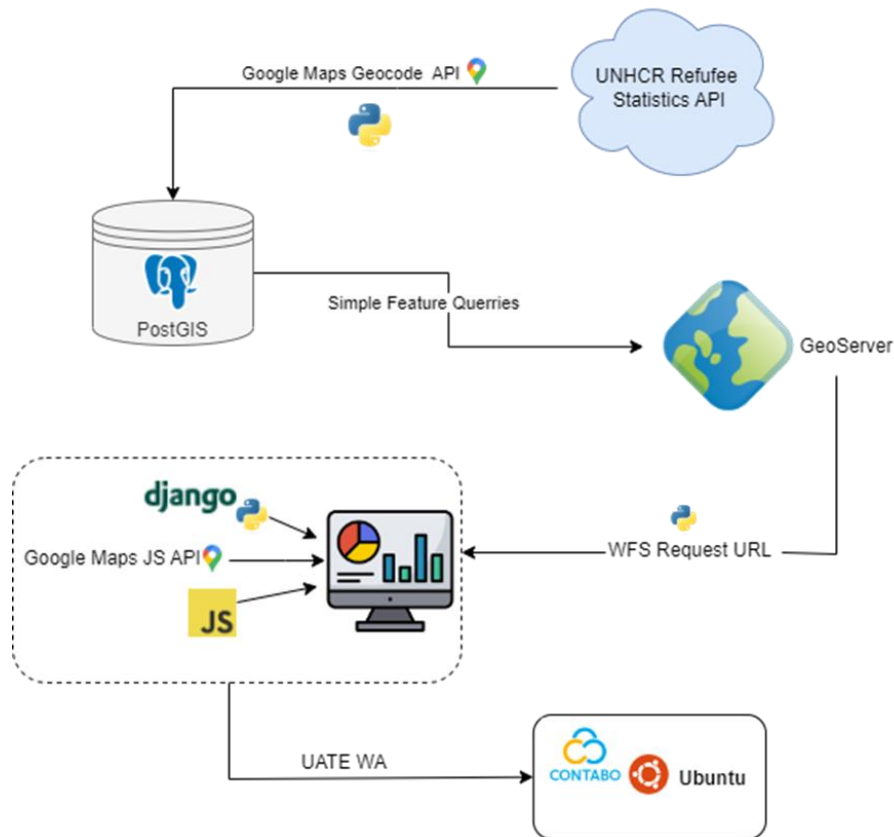


Figure 5: UATE SDI architecture diagram

WP DATA COLLECTION AND INTEGRATION

A deeper examination and exploration of the data was carried out in this work package with the objective of cleaning it to make it suitable for the project needs. As mentioned in the previous section, the refugee statistics API data endpoints that were of interest to the project include:

- **Asylum applications** - the dataset contained information on the year, number of applicants, the type of application such as new or appeals.
- **Asylum decisions** - provided information for year, number of decisions, decision level and type of decision such as rejected or closed.
- **Demographics** – provided population breakdown of Ukrainians seeking and living with asylum status in the respective countries.

The metadata retrieved from the API was very limited. For supplementation, a documentation on how the data was collected and key abbreviations were derived from a handbook that was available on the UNHCR refugee statistics website. Additionally, the website also provides a description on what the data represents which was useful in determining what elements of that data were to be used in the visualization dashboard.

The data was then cleaned for consistency by careful elimination of duplicates by merging and filtering the type of applications and decisions that would be considered for the project.

Additionally, for the demographic's dataset, only Germany recorded the population breakdown in terms of age groups in the sample countries used for testing. Therefore, for uniformity in the project, the population totals were limited to total males and females regardless of their age.

a) Geocoding the data

The retrieved data lacked geographical attributes but had ISO codes and name for each country. Therefore, geocoding was required for all the retrieved datasets. The Google Geocoding API was included in the script to geocode the data before writing it to the database. This was the final data tuning for the API endpoints. The geocoding converted the country name to points and added it as a column in the in the database. A snippet of the code is shown below:

```
# function to get coordinates for a country name using Google Geocoding API
def get_coordinates(country_name, api_key):
    geo_url = "https://maps.googleapis.com/maps/api/geocode/json"
    params = {"address": country_name, "key": api_key}
    response = requests.get(geo_url, params=params)
    if response.status_code == 200 and response.json()['results']:
        result = response.json()["results"][0]["geometry"]["location"]
        return result["lat"], result["lng"]
    else:
        return None, None
```

Figure 6: Code snippet of geocoding function in Django views.py

b) Database connection

To write the data to the database, a connection to the PostGIS database was established using the *psycopg2* library to allow *SQL* queries to be directly made from the script to the database.

```
# Database credentials
db_params = {
    "dbname": " your db name",
    "user": " user",
    "password": "your password",
    "host": "Your host"}

# Connect to the PostgreSQL database
conn = psycopg2.connect(**db_params)
cursor = conn.cursor()
```

Figure 7: Database connection using *psycopg2*

WP 4 GEOSPATIAL SERVICES AND MAPPING

a) GeoServer integration

The three datasets were now stored in the database and ready to be hosted for consumption. For this purpose, an open-source option in GeoServer was sought. A connection was made to the PostGIS database from GeoServer through simple feature queries and the tables for the respective datasets were published to allow for external client access. The data was made available as a web feature service (WFS) and a web map service (WMS).

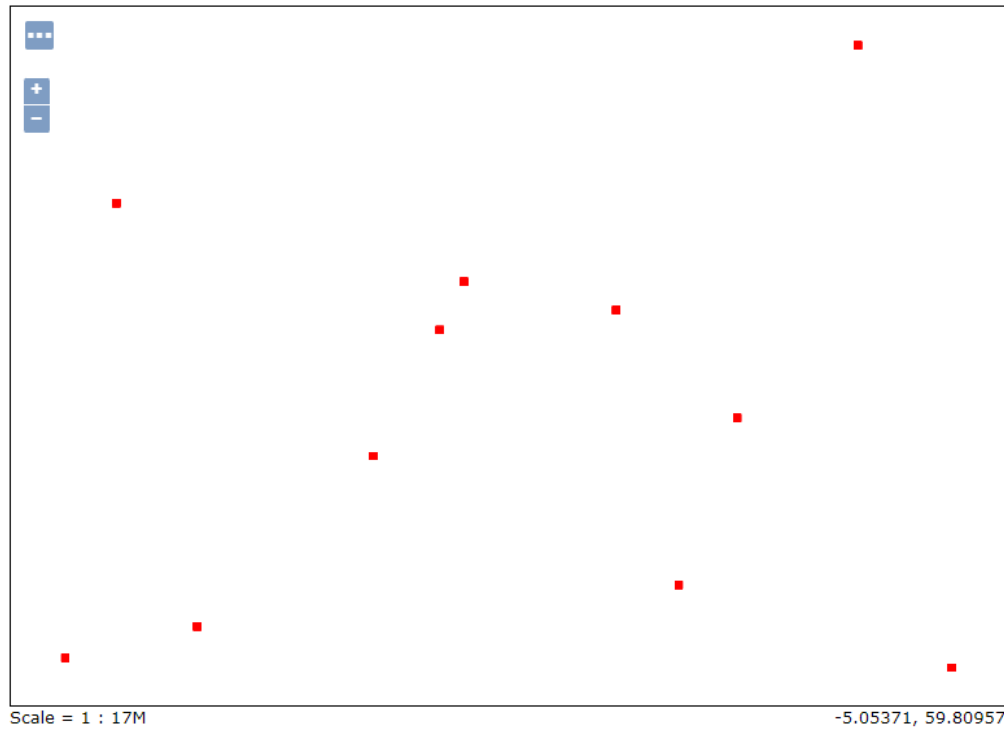


Figure 8: Screenshot showing published data viewed with open layers in GeoServer

b) Setting up Django project and accessing the data

The data infrastructure was consumed as a web application/ dashboard built on the Django Python framework. After setting up a Django project and app, in the *views.py* file, a function was created to fetch and process data related to asylum decisions, applications, and Ukrainian demographics from GeoServer based on year-specific queries. It also transforms raw JSON data

into structured dictionaries for easier handling, creating lookup tables and formatted lists for mapping and visualization purposes.

```
def transform_applications_to_array(json_data):  
    # Extract the features from the JSON data  
    features = json_data.get('features', [])  
    transformed_array = []  
    # Extract relevant information  
    for feature in features:  
        properties = feature.get('properties', {})  
        geometry = feature.get('geometry', {}).get('coordinates', [])  
  
        transformed_object = {  
            'year': properties.get('year'),  
            'coo_name': properties.get('coo_name'),  
            'coa_name': properties.get('coa_name'),  
            'app_type': properties.get('app_type'), # New property  
            'applied': properties.get('applied'),  
            'coordinates': geometry,  
        }  
        # Add the transformed object to the array  
        transformed_array.append(transformed_object)  
    return transformed_array
```

Figure 9: Retrieved data processing code snippet in Django views.py

c) Mapping library integration

The Maps JavaScript API was the primary choice for a mapping library, it was integrated to visualize the features retrieved from the *views.py* file. To achieve this, a templates folder was created in the Django project to host the html files. The mapping library was integrated in the *home.html* file.

```
{% extends 'uate.html' %} {% load static %} {% block content %}
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Map of Selected Countries</title>
    <script
      src="https://maps.googleapis.com/maps/api/js?key={{GOOGLE_API_KEY}}&callback
=initMap"
      async
      defer
    ></script>
  </head>
</html>
```

Figure 10: Snippet showing Google Maps JavaScript API integration in the project's HTML templates

WP DASHBOARD DEVELOPMENT

When the project setup was complete with the data retrieval and mapping integration, it was time to optimize the Django project by adding necessary files and linking them together. This included adding all the stylesheets, template files and library installations.

a) Origin-destination visualization

An implementation had to be done to visualize the origin and destination countries using the Google maps API polyline function as show below:

```
var currentLine = null;
function drawLineToUkraine(coaCoords) {
    if (currentLine) {
        // Remove existing line if present
        currentLine.setMap(null);
    }
    var lineSymbol = {path: google.maps.SymbolPath.FORWARD_CLOSED_ARROW};
    var line = new google.maps.Polyline({
        path: [ukraineCoords, coaCoords],
        icons: [{icon: lineSymbol,
            offset: '0%'}],
        map: map,
        geodesic: true,
        strokeColor: "red",
        strokeOpacity: 1.0,
        strokeWeight: 2,
    });
    // Update current line
    currentLine = line;
    // Call the function to animate the arrow
    animateArrow(line);
}
```

Figure 11: Google Maps Polyline function to visualize origin - destinations on the map

b) Sample chart

To visualize the data retrieved, the Chart JS library was used to create line graphs, bar and pie charts for the data.

```
// Create chart for opulation using chart JS
var pchartData = {
  labels: ["Female", "Male"],
  datasets: [
    {
      data: [
        Math.round((data.f_total / (data.f_total + data.m_total)) * 100),
        Math.round((data.m_total / (data.f_total + data.m_total)) * 100),
      ],
      backgroundColor: ["rgba(255, 99, 132, 0.5)", "rgba(54, 162, 235, 0.5)"],
      borderColor: ["rgba(255, 99, 132, 1)", "rgba(54, 162, 235, 1)"],
      borderWidth: 1,
    },
  ],
};
```

Figure 12: Snippet showing use of chart.js library for data visualization in the dashboard

c) User interaction implementation

To add user interactivity with the country markers, charts and information pens, JavaScript came in handy by listening to and acting on user actions on the dashboard.

```
function showInfoContent1() {
  let infoContent1 = document.getElementById("infoContent1");
  let infoContent2 = document.getElementById("infoContent2");
  let infoBtn1 = document.getElementById("infoBtn1");
  let infoBtn2 = document.getElementById("infoBtn2");
  infoContent2.classList.add("hidden");
  infoContent1.classList.remove("hidden");
  infoBtn1.classList.add("bg-[#1f2a38]");
  infoBtn2.classList.remove("bg-[#1f2a38]");
}
```

Figure 13: JavaScript snippet for facilitating user interactivity in the dashboard

d) Final dashboard

The dashboard can be found here <http://uate.lipiangoma.co.ke/>.

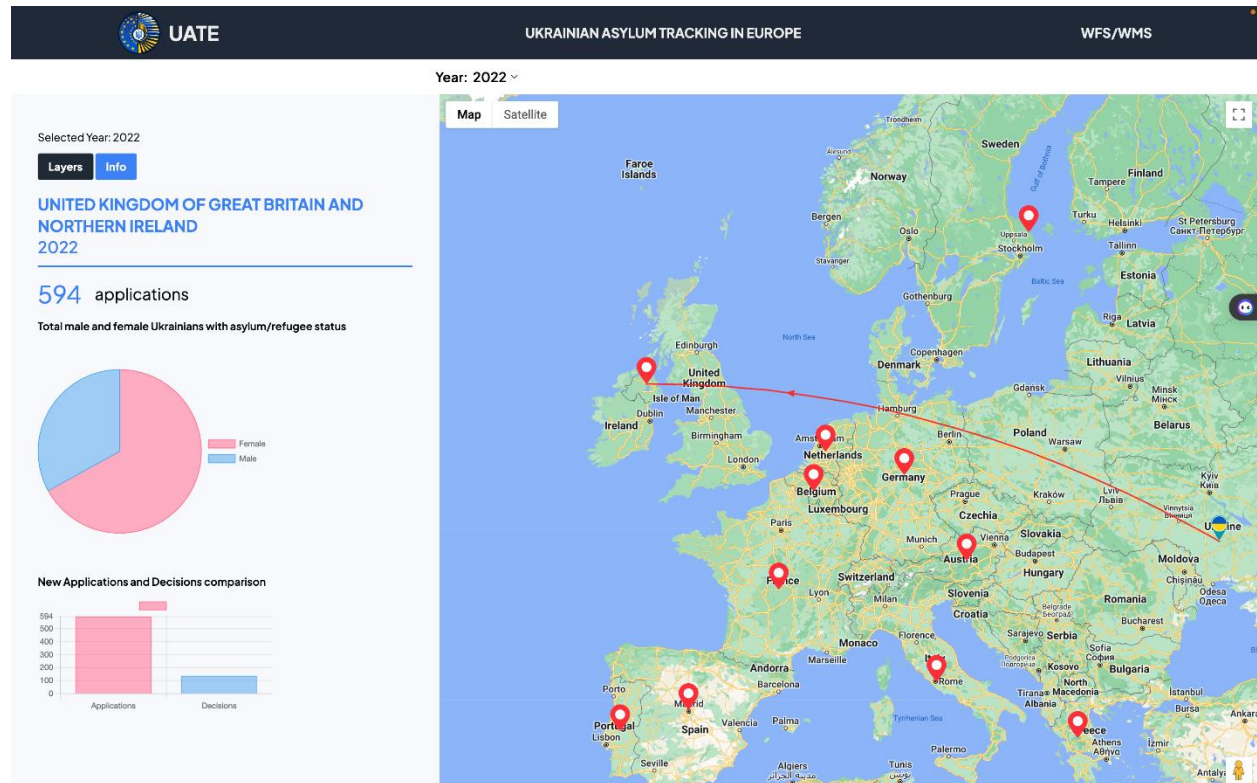


Figure 14: Screenshot showing data visualization on the dashboard

e) Deployment

The Django project was deployed on a free tier Amazon Web Services (AWS) instance utilizing a Contabo Ubuntu server. This approach leveraged the cost-efficiency and scalability of AWS's cloud services alongside the reliability and performance of Contabo's virtual servers.

CHALLENGES

- **Data structure** of the UNHCR refugee statistics API data posed significant challenges and required extensive cleaning and conversions before writing it to the database and publishing in GeoServer.
- **Data consistency** was also a challenge. A good example was with the recording of demographic breakdown whereby only Germany had recorded the breakdown. However, the discrepancy may be attributed to the policies of individual countries submitting the data.

- **Scalability** could not be achieved due to the data retrieval limitations from the UNHCR refugee statistics API.

OUTLOOK

There are avenues for improvement that could be explored for the UATE SDI by incorporating other migration data sources to enrich the infrastructure.